Netzwerke 1

Abgabe 1

Melvin Lux, Angelina Vorndran

Bachelor Informatik, 3. Semester

Fakultät Informatik und Mathematik

Hochschule München University of Applied Sciences

München, 6. November 2026

1 Blatt 1

Aufgabe 1.1: Netzwerk-Traces mit Wireshark

Teilaufgabe a) Aufruf der Webseite www.cs.hm.edu im Browser

- **Beobachtete Protokolle:** DNS, TCP, UDP, TLSv1.3, QUIC
- **Ablauf & Beschreibung:**
  1. **DNS (Domain Name System):** Löst den Domainnamen `www.cs.hm.edu` in die zugehörige IP-Adresse auf (typischerweise über UDP).
  2. **TCP / UDP:** Transportprotokolle für den Verbindungsaufbau und die Datenübertragung.
  3. **TLSv1.3 / QUIC:** Sichere, verschlüsselte Protokolle zur Übertragung von HTTPS-Webinhalten. QUIC setzt dabei auf UDP auf und kombiniert Verbindungsaufbau sowie Verschlüsselung.

Teilaufgabe b) Ping auf www.google.com

- **Beobachtete Protokolle:** DNS, ICMP / ICMPv6
- **Antwortende IP-Adresse:** `142.251.153.119`
- **Ablauf & Beschreibung:**
  1. **DNS:** Ermittelt die IP-Adresse von `www.google.com`.
  2. **ICMP (Internet Control Message Protocol):** Sendet *Echo Request*-Pakete an die Ziel-IP. Der Rechner antwortet mit *Echo Reply*-Paketen zur Diagnose der Erreichbarkeit und Latenz.

Teilaufgabe c) FTP-Verzeichnisanzeige (ftp.slackware.com)

- **Beobachtete Protokolle:** DNS, TCP, FTP
- **Anmeldedaten (Anonymous-FTP):**
  - Benutzername: `ftp` oder `anonymous`
  - Passwort: z. B. eine E-Mail-Adresse wie `ftp@melvinlux.de`
- **Erzielte Datenrate:** ca. 11,59 MiB/s
- **Sicherheitsproblem von FTP:** FTP überträgt Steuerdaten (einschließlich Benutzername und Passwort im Klartext) sowie alle Nutzdaten unverschlüsselt. Ein Angreifer im selben

Netzwerk kann diese sensiblen Daten problemlos über Wireshark mitlesen.

Einordnung der Protokolle in die Schichten (OSI-Modell)

Protokoll	Schicht im OSI-Modell	Bezeichnung der Schicht
ARP	Schicht 2 / 3	Data Link / Network
ICMP	Schicht 3	Network
TCP / UDP	Schicht 4	Transport
TLSv1.3	Schicht 5	Session
QUIC	Schicht 5 + 6	Session & Presentation
DNS, FTP, NTP	Schicht 7	Application

Tabelle 1: Einordnung der beobachteten Netzwerkprotokolle in das OSI-Referenzmodell.

Aufgabe 1.2: Netzwerkkommunikation ganz ohne Netzwerkprogrammierung

1. Datei von Client zu Server senden

- **Server-Seite:**

```
nc -k -l 4711 > received.txt
```

Erklärung: Der Server lauscht auf Port 4711 (`-l`), bleibt nach Verbindungsende aktiv (`-k`) und leitet alle eingehenden Daten in die Datei `received.txt` um (`>`).

- **Client-Seite:**

```
cat test.txt | nc localhost 4711
```

Erklärung: Der Inhalt der Datei `test.txt` wird über die Pipe (`|`) an den Client `nc` übergeben und an den lokalen Port 4711 gesendet.

2. Netzwerkdienst zur Umwandlung von Texten

- **Server-Seite:**

```
nc -l 4711 < pipe | sed -u 's/Vorlesung/Party/g' > pipe
```

Erklärung:

1. `mkfifo pipe`: Erstellt eine benannte Pipeline (Named Pipe), um eine Rückkopplung/Schleife für Daten zu ermöglichen.
2. `nc -l 4711 < pipe`: Liest die verarbeitete Antwort aus der Named Pipe und schickt sie zurück an den Client.
3. `| sed -u 's/Vorlesung/Party/g'`: Der Stream wird durch `sed` verarbeitet. Jedes Vorkommen von `"Vorlesung"` wird global (`/g`) durch `PParty"` ersetzt. Die Option `-u` (*unbuffered*) stellt sicher, dass der Text sofort ohne Pufferung ausgegeben wird.
4. `> pipe`: Schreibt das Ergebnis zurück in die Named Pipe.

- **Client-Seite:**

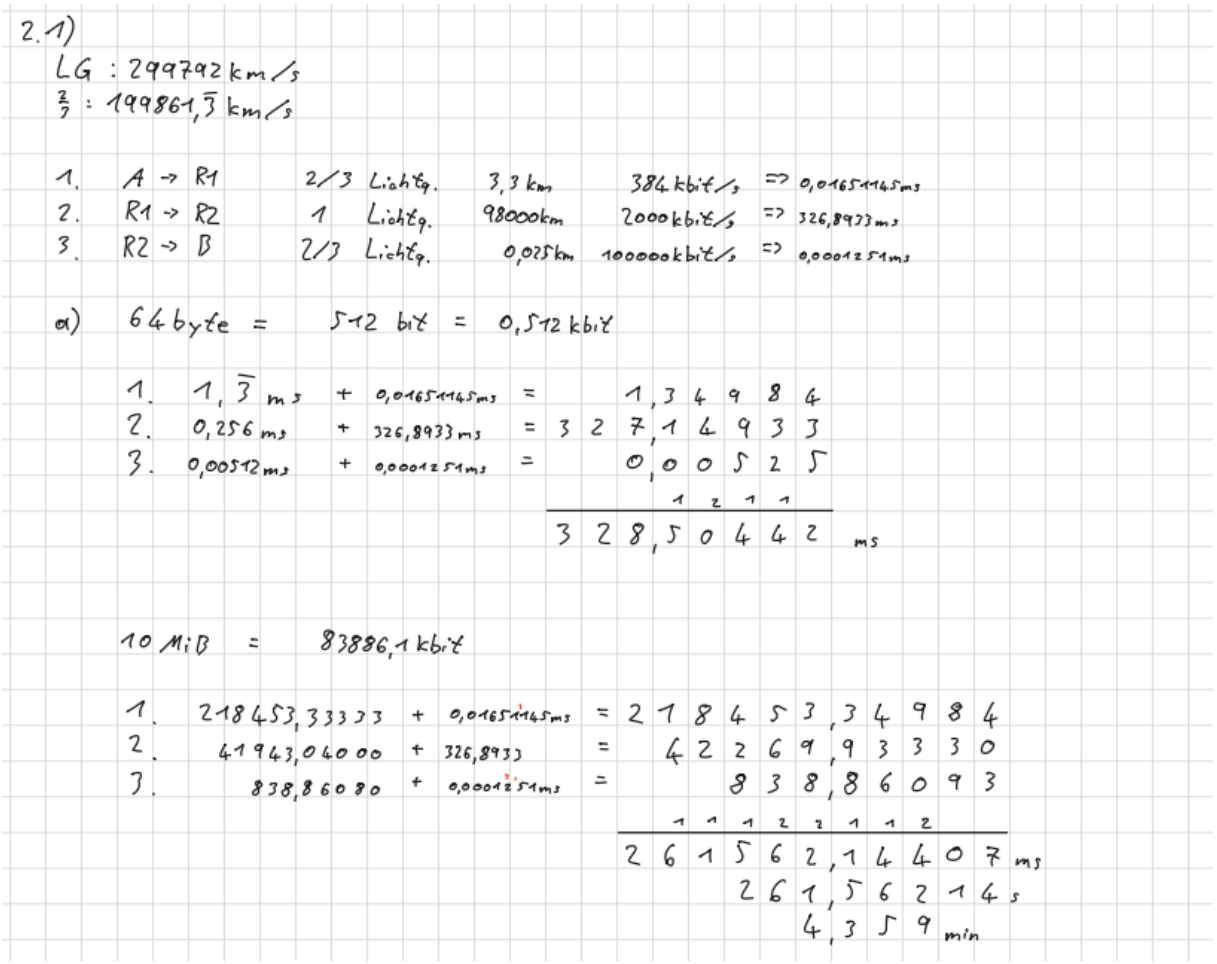
```
nc localhost 4711
```

Erklärung: Baut eine interaktive Verbindung zum Dienst auf Port 4711 auf. Nach der Eingabe von z. B. `Ist morgen Vorlesung?` empfängt der Client die Antwort `Ist morgen Party?`.

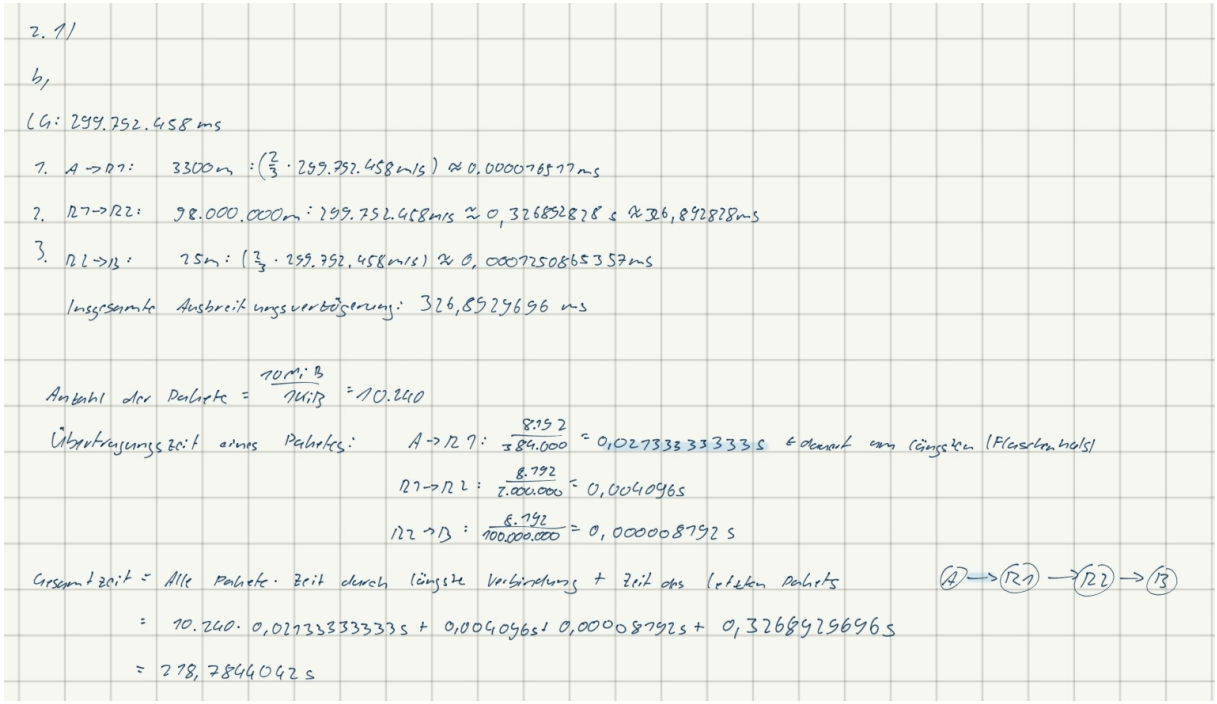
2 Blatt 2

2.1 Aufgabe 2.1

2.1.1 a



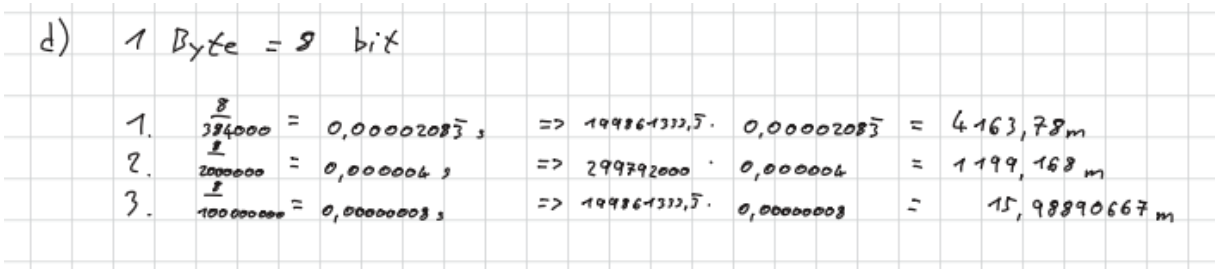
2.1.2 b



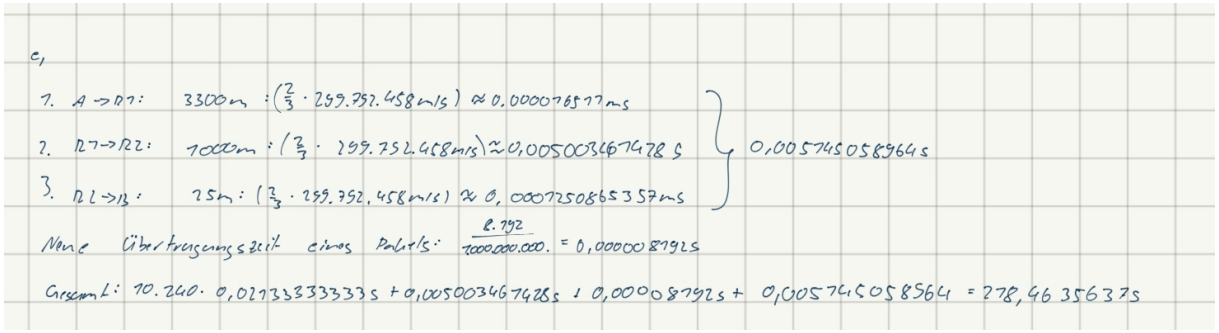
2.1.3 c

Wenn man eine der Variablen verändern könnte würden wir die Übertragungsrate vom 1. Schritt (A->R1) erhöhen um die dortigen Zeiten drastisch zu erhöhen.

2.1.4 d



2.1.5 e



2.2 Aufgabe 2.2

2.2.1 a

Das Internet-Schichtenmodell unterscheidet sich vom ISO/OSI-Schichtenmodell hauptsächlich an 2 Stellen, indem es einige Schichten zusammenfasst:

- 1. Verbindungsschicht -> Bitübertragungsschicht & Sicherungsschicht
- 2. Anwendungsschicht -> Kommunikationssteuerung & Darstellungsschicht & Anwendungsschicht

Im Allgemeinen lässt sich sagen, dass das Internet-Schichtenmodell das ISO/OSI-Schichtenmodell simplifiziert und für den Endanwender unnötige Schichten zusammenfasst.

2.2.2 b

Funktionalität wird im Internet immer auf der höchstmöglichen Schicht realisiert, um keine neue Hardware in der Infrastruktur zu benötigen, die Netzneutralität nicht zu gefährden und um weitere Kosten durch den Netzbetreiber vermeiden zu können.

2.2.3 c

Nein. Ein zuverlässiges Schicht-2-Protokoll garantiert die korrekte Übertragung nur zwischen zwei direkt verbundenen Knoten, also pro Teilstrecke. Auf dem Weg zwischen den Endsystemen können Pakete trotzdem verloren gehen (z. B. durch überlaufende Router-Puffer), verfälscht werden (Fehler im Router nach der Link-Prüfung), dupliziert werden oder in falscher Reihenfolge ankommen. Diese Fehler werden auf Schicht 2 nicht erkannt. Zuverlässigkeit muss

daher Ende-zu-Ende gewährleistet werden, also im Transportprotokoll (TCP) oder in der Anwendung. UDP bietet das nicht. Der gesparte Overhead würde die Zuverlässigkeit gefährden.

2.3 Aufgabe 2.3

2.3.1 HelloServer

Der Vollständige Code ist in HelloServer.java zu finden.

2.3.2 FreeDiskSpaceClient

Der Vollständige Code ist in FreeDiskSpaceClient.java zu finden.

3 Blatt 3

3.1 3.1

3.1.1 a

$https://cs.hm.edu/aktuelles/news/news_detailseite_133888.de.html$

3.1.2 b

Ja, damit kann umgegangen werden, da im Header Äccept-Encoding"gzip enthalten ist.

3.1.3 c

Es wird eine persistente Verbindung erwartet, da der Header Connection"den Wert keep-alive hat.

3.1.4 d

Wenn der Header Ïf-None-Match"der Server würde ausschließlich Datein zurückgeben die den ETag "ddnenj89j4jf8ejf8juw37EEjh8Suf3nnicht besitzen, dann würde auch der Code 200 gesendet werden, wenn eine solche Datei existiert bekommen wir 304 NNot Modifiedßurück.

3.1.5 e

Der Server kann mit 2 Möglichkeiten antworten:

- 200 — OK

```
❯ curl --compressed "https://cantamus.app/packages/verovio-3.3.0/verovio.js" -H "If-None-Match: f1ee2cc0ca9c51
f2d05575f8c1436407-ssl-df" -I
HTTP/2 200
accept-ranges: bytes
cache-control: public, max-age=0, must-revalidate
content-length: 7442291
content-type: application/javascript
date: Tue, 18 May 2021 14:18:06 GMT
etag: "f1ee2cc0ca9c51f2d05575f8c1436407-ssl"
strict-transport-security: max-age=31536000
age: 4437
x-nf-request-id: 34914077-fc7f-4bf1-82eb-06852a4801d6
server: Netlify
```

- 304 — Not Modified

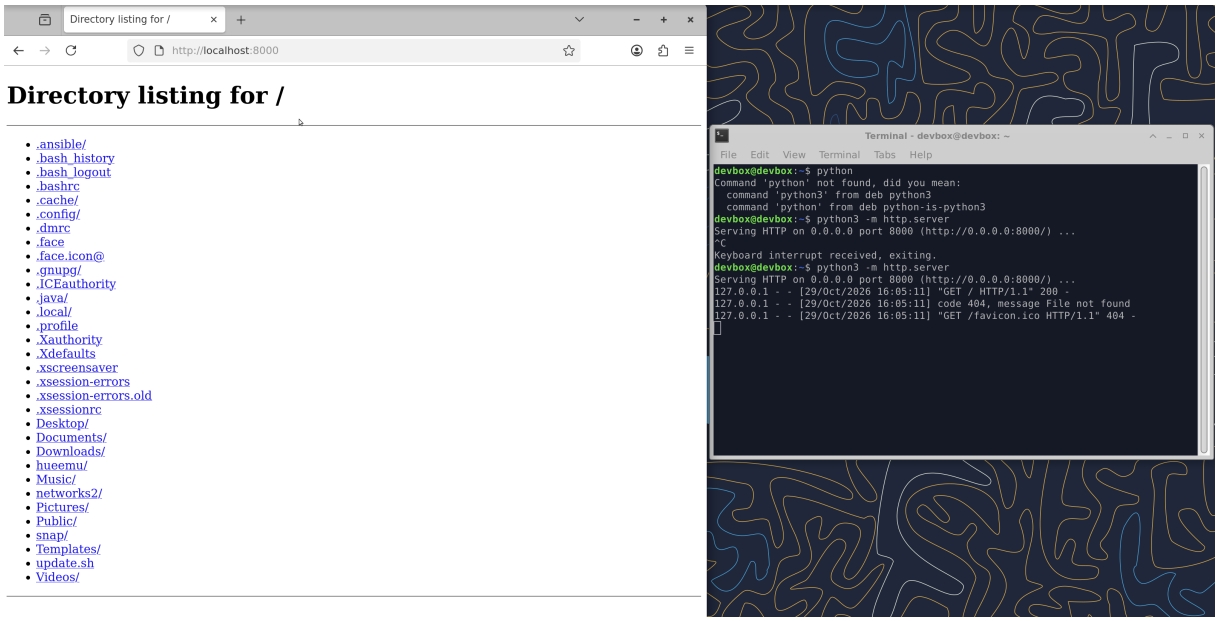
```
❯ curl --compressed "https://cantamus.app/packages/verovio-3.3.0/verovio.js" -H "If-None-Match: f1ee2cc0ca9c51
f2d05575f8c1436407-ssl-df" -I
HTTP/2 304
date: Tue, 18 May 2021 15:32:04 GMT
etag: "f1ee2cc0ca9c51f2d05575f8c1436407-ssl-df"
cache-control: public, max-age=0, must-revalidate
x-nf-request-id: 4aad8a5b-8368-4716-8ca2-9f9018a749b2
server: Netlify
vary: Accept-Encoding
```

3.2 3.2

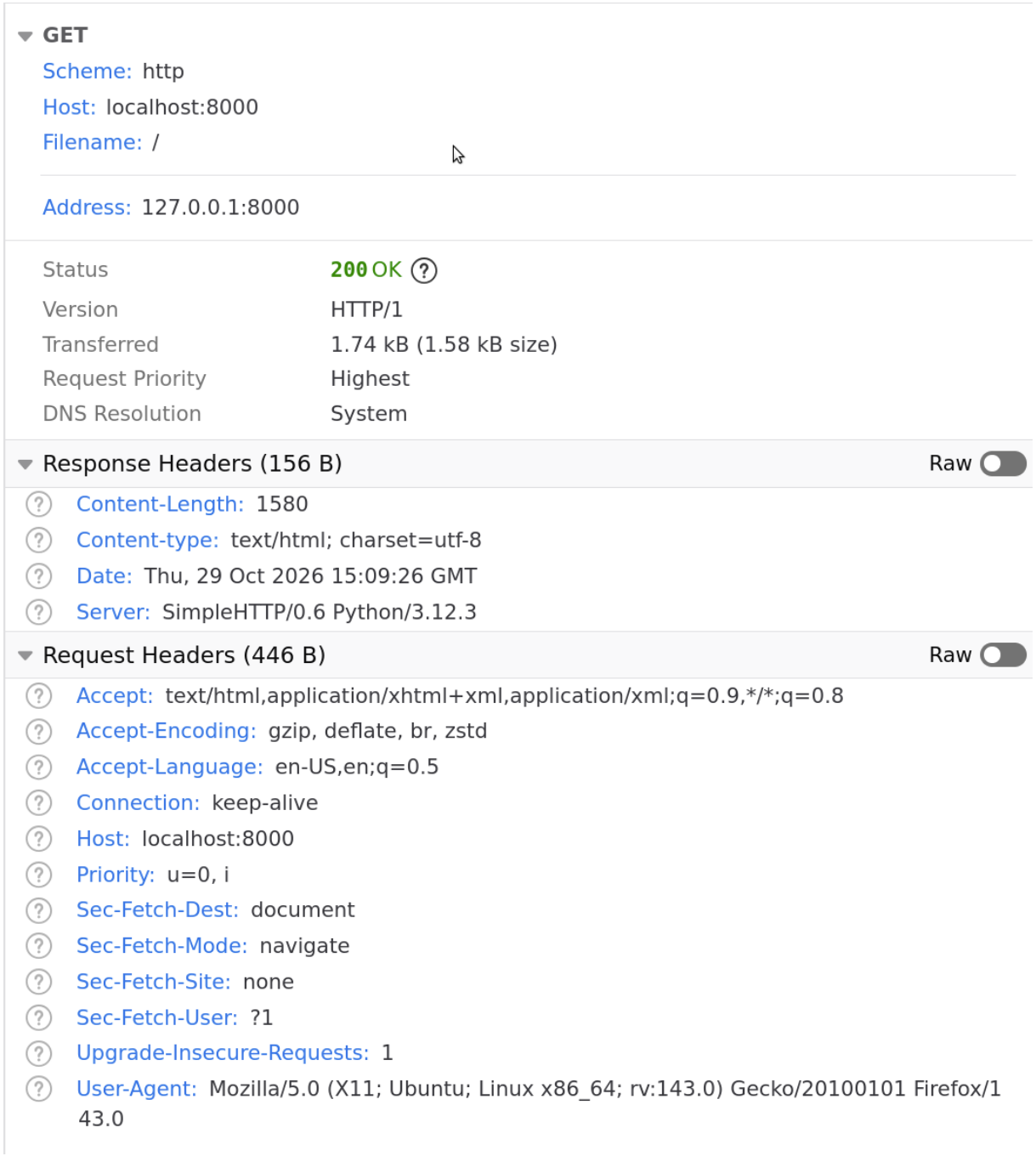
3.3 3.3

3.3.1 a

Es werden die Datein aus dem Ordner angezeigt, indem der Befehl ausgeführt wurde, hier das Homeverzeichnis.


Directory listing for /
http://localhost:8000
Directory listing for /
.ansible/
.bash_history
.bash_logout
.bashrc
.cache/
.config/
.dmrc
.face
.face.icon@
.gnupg/
.ICEauthority
.java/
.local/
.profile
.Xauthority
.Xdefaults
.xscreensaver
.xsession-errors
.xsession-errors.old
.xsessionrc
Desktop/
Documents/
Downloads/
hueemu/
Music/
networks2/
Pictures/
Public/
snap/
Templates/
update.sh
Videos/
Terminal - devbox@devbox: ~
File Edit View Terminal Tabs Help
devbox@devbox:~$ python
Command 'python' not found, did you mean:
command 'python3' from deb python3
command 'python' from deb python-is-python3
devbox@devbox:~$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
^C
Keyboard interrupt received, exiting.
devbox@devbox:~$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [29/Oct/2026 16:05:11] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Oct/2026 16:05:11] code 404, message File not found
127.0.0.1 - - [29/Oct/2026 16:05:11] "GET /favicon.ico HTTP/1.1" 404 -

3.3.2 b



3.3.3 c

Der Vollständige Code ist in OwnClient_v1_0.java zu finden.

3.3.4 d

Ab HTTP 1.1 wird der Header Ḧostöffiziell als verpflichtend gekennzeichnet, in v1.0 war dieser schon möglich, jedoch nicht verlangt. Viele Services erwarten diesen Header jedoch unabhängig vom Protokoll, da der Header dem Anbieter ermöglicht Anfragen durch Proxies und Loadbalancers zu routen ohne, dass das Ziel verloren geht. Außerdem ermöglicht dieser Header dem Webserver unterschiedliche Websites zu liefern ohne benötigt manuelle navigation.

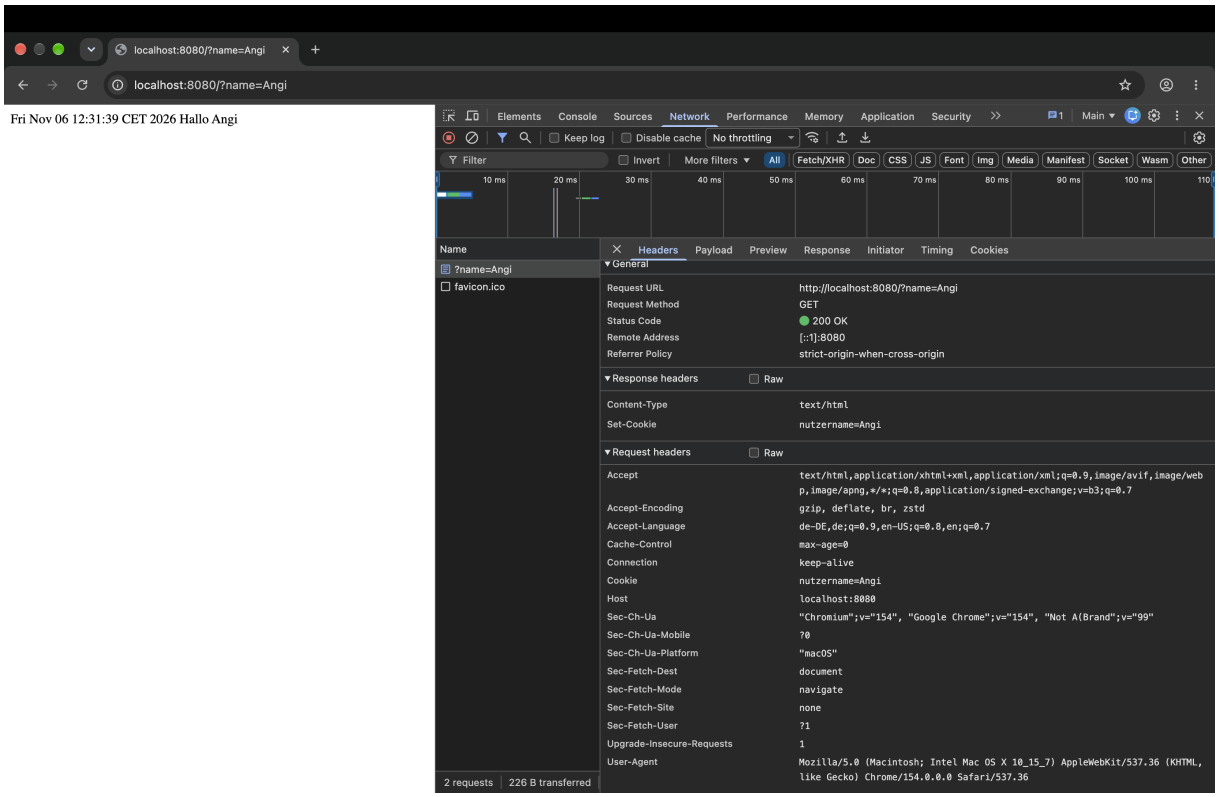
3.3.5 e

Der Vollständige Code ist in OwnClient_v1_1.java zu finden.

Aus Sicht des Developers ändert sich das man nur noch eine Variable / Konstate hat für eine mehrteilige URL, außerdem verringert sich die Möglichkeit Fehler zu machen beim Aufrauf einer URL.

3.4 3.4

Der Vollständige Code ist in HelloServer.java zu finden.



3.5 3.5

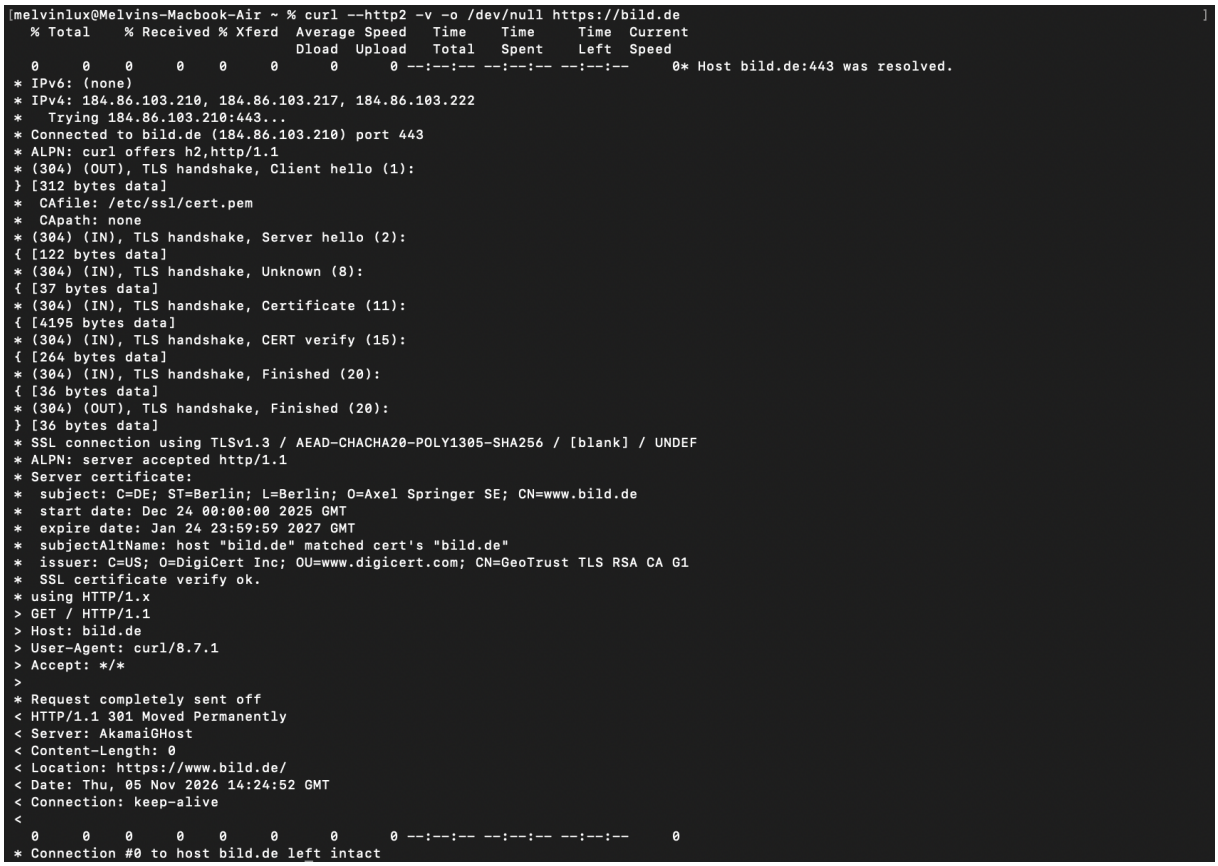
3.5.1 a

•

```
melvinlux@Melvins-Macbook-Air ~ % curl --http2 -v -o /dev/null https://www.hm.edu
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Host www.hm.edu:443 was resolved.
* IPv6: 2001:4ca0:6800:3::2
* IPv4: 129.187.244.122
*   Trying [2001:4ca0:6800:3::2]:443...
* Connected to www.hm.edu (2001:4ca0:6800:3::2) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
} [315 bytes data]
*  CAfile: /etc/ssl/cert.pem
*  CApath: none
* (304) (IN), TLS handshake, Server hello (2):
{ [122 bytes data]
* (304) (IN), TLS handshake, Unknown (8):
{ [19 bytes data]
* (304) (IN), TLS handshake, Certificate (11):
{ [3407 bytes data]
* (304) (IN), TLS handshake, CERT verify (15):
{ [79 bytes data]
* (304) (IN), TLS handshake, Finished (20):
{ [36 bytes data]
* (304) (OUT), TLS handshake, Finished (20):
} [36 bytes data]
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted h2
* Server certificate:
*  subject: CN=www.hm.edu
*  start date: Sep 20 05:52:51 2026 GMT
*  expire date: Dec 19 05:52:50 2026 GMT
*  subjectAltName: host "www.hm.edu" matched cert's "www.hm.edu"
*  issuer: C=US; O=Let's Encrypt; CN=YE1
*  SSL certificate verify ok.
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://www.hm.edu/
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: www.hm.edu]
* [HTTP/2] [1] [:path: /]
* [HTTP/2] [1] [user-agent: curl/8.7.1]
* [HTTP/2] [1] [accept: */*]
> GET / HTTP/2
> Host: www.hm.edu
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/2 302
< access-control-allow-methods: GET
< access-control-allow-origin: *
< alt-svc: h3=":443"; ma=2592000
< content-security-policy: img-src imagedelivery.net hm.vostok.zone35.net *.b-ite.com *.hm.edu flockler.com fl-1.cdn.flockler.com social-proxy.flocklr.com media-api.flockler.com media.licdn.com cloud.ccm19.de *.cdninstagram.com *.xx.fbcdn.net 'self' data:; font-src *.assisto.beranet.de formulare.hm.edu formulare-test.hm.edu assets.hm.edu mediapool.hm.edu mediapool-prem.hm.edu data:; script-src *.assisto.beranet.de *.b-ite.com plugins.flockler.com matomo.hm.edu assets.hm.edu mediapool.hm.edu mediapool-prem.hm.edu cloud.ccm19.de 'unsafe-inline'; script-src-elem *.assisto.beranet.de *.b-ite.com plugins.flockler.com matomo.hm.edu formulare.hm.edu formulare-test.hm.edu assets.hm.edu mediapool.hm.edu mediapool-prem.hm.edu cloud.ccm19.de 'unsafe-inline' 'self'; script-src-attr 'unsafe-inline'; connect-src wss://hm.vostok.zone35.net cdn.jsdelivr.net hm.vostok.zone35.net *.beranet.de hm-edu-search-api.e-spirit.cloud *.b-ite.com stats-api.flockler.app api.flockler.app *.hm.edu cloud.ccm19.de matomo.hm.edu; style-src *.assisto.beranet.de *.b-ite.com assets.hm.edu mediapool.hm.edu mediapool-prem.hm.edu cloud.ccm19.de 'unsafe-inline'; style-src-elem *.assisto.beranet.de *.b-ite.com formulare.hm.edu formulare-test.hm.edu mediapool.hm.edu mediapool-prem.hm.edu assets.hm.edu cloud.ccm19.de 'unsafe-inline'; style-src-attr 'unsafe-inline'; default-src 'self' plugins.flockler.com matomo.hm.edu search.hm.edu formulare.hm.edu formulare-test.hm.edu cloud.ccm19.de assets.hm.edu mediapool.hm.edu mediapool-prem.hm.edu 'unsafe-inline'; media-src 'self' dms.licdn.com media-api.flockler.com data:; frame-src 'self' cloud.ccm19.de mstream.hm.edu video.hm.edu www.youtube.com www.youtube-nocookie.com media-api.flockler.com *.cloudflarestream.com; child-src 'self'; frame-ancestors 'none'; base-uri 'self'; worker-src 'none'; manifest-src 'none'; report-to https://sentry.hm.edu/api/9/security/?sentry_key=3d6f839d908181cf6622e6f584d2efe3; report-uri https://sentry.hm.edu/api/9/security/?sentry_key=3d6f839d908181cf6622e6f584d2efe3;
< location: https://hm.edu/
< permissions-policy: accelerometer=(), autoplay=(), camera=(), cross-origin-isolated=(), display-capture=(), encrypted-media=(), fullscreen=(self), geolocation=(), gyroscope=(), keyboard-map=(), magnetometer=(), microphone=(), midi=(), payment=(), picture-in-picture=(self), publickey-credentials-get=(), screen-wake-lock=(), sync-xhr=(), usb=(), xr-spatial-tracking=()
< referrer-policy: strict-origin
< strict-transport-security: max-age=31536000
< x-content-type-options: nosniff
< x-frame-options: SAMEORIGIN
< x-xss-protection: 1; mode=block
< content-length: 0
< date: Thu, 05 Nov 2026 14:23:19 GMT
<
{ [0 bytes data]
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
* Connection #0 to host www.hm.edu left intact
```

HTTP2 wird unterstützt und per ALPN negotiatet

•



HTTP2 wird nicht unterstützt und per ALPN wird HTTP1.1 ausgehandelt

•

```
melvinlux@Melvins-Macbook-Air ~ % curl --http2 -v -o /dev/null https://linux.de
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0* Host linux.de:443 was resolved.
* IPv6: (none)
* IPv4: 85.214.247.117
*   Trying 85.214.247.117:443...
* Connected to linux.de (85.214.247.117) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
} [313 bytes data]
*  CAfile: /etc/ssl/cert.pem
*  CApath: none
* (304) (IN), TLS handshake, Server hello (2):
{ [122 bytes data]
* (304) (IN), TLS handshake, Unknown (8):
{ [25 bytes data]
* (304) (IN), TLS handshake, Certificate (11):
{ [4087 bytes data]
* (304) (IN), TLS handshake, CERT verify (15):
{ [264 bytes data]
* (304) (IN), TLS handshake, Finished (20):
{ [36 bytes data]
* (304) (OUT), TLS handshake, Finished (20):
} [36 bytes data]
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted http/1.1
* Server certificate:
*  subject: CN=linux.de
*  start date: Oct 29 14:00:41 2026 GMT
*  expire date: Jan 27 14:00:40 2027 GMT
*  subjectAltName: host "linux.de" matched cert's "linux.de"
*  issuer: C=US; O=Let's Encrypt; CN=YR2
*  SSL certificate verify ok.
* using HTTP/1.x
> GET / HTTP/1.1
> Host: linux.de
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Date: Thu, 05 Nov 2026 14:26:07 GMT
< Server: Apache
< Location: https://www.linux.de/
< Content-Length: 288
< Content-Type: text/html; charset=iso-8859-1
<
{ [288 bytes data]
100   288  100   288    0     0   1535      0 --:--:-- --:--:-- --:--:--  1540
* Connection #0 to host linux.de left intact
```

HTTP2 wird nicht unterstützt und ALPN handelt HTTP1.1 aus

3.5.2 b

•

```
melvinlux@Melvins-Macbook-Air NW1 % curl --http1.0 -w "@format.txt" -o /dev/null "https://www.google.com/search?q=krautsalat&rlz=1C5CHFA_enDE1156DE1156&oq
=krautsalat&gs_lcrp=EgZjaHJvbWUqBwgAEAAYjwIyBwgAEAAYjwIyCggBEC4YsQMYgAQyCggCEAAYsQMYgAQyBwgDEAAYgAQyBwgEEAAYgAQyBwgFEAAYgAQyBwgGEAAYgAQyBwgHEAAYgAQyBwgIEA
AYgAQyBwgJEAAYjwLSAQgxOTAwajBqN6gCALACAA&sourceid=chrome&source=chrome.ob&ie=UTF-8"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 92903    0 92903    0     0   517k      0 --:--:-- --:--:-- --:--:--  515k
benutzte HTTP Version: 1
 Request : 0 bytes
 Response : 92903 bytes
 Geschwindigkeit : 529935 bytes/s
 Dauer DNS Anfrage : 0.020925 s
 Dauer TCP Aufbau : 0.038981 s
 Dauer TLS Handshake : 0.067650 s
 Gesamtzeit : 0.175310 s
```

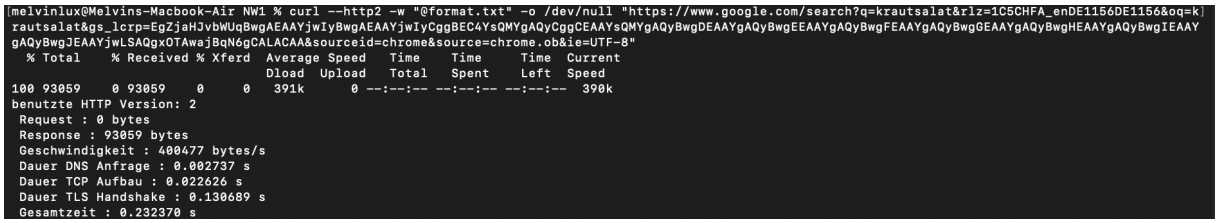
HTTP1.0

•

```
melvinlux@Melvins-Macbook-Air NW1 % curl --http1.1 -w "@format.txt" -o /dev/null "https://www.google.com/search?q=krautsalat&rlz=1C5CHFA_enDE1156DE1156&oq]
=krautsalat&gs_lcrp=EgZjaHJvbWUqBwgAEAAYjwIyBwgAEAAYjwIyCggBEC4YsQMYgAQyCggCEAAYsQMYgAQyBwgDEAAYgAQyBwgEEAAYgAQyBwgFEAAYgAQyBwgGEAAYgAQyBwgHEAAYgAQyBwgIEA
AYgAQyBwgJEAAYjwLSAQgxOTAwajBqN6gCALACAA&sourceid=chrome&source=chrome.ob&ie=UTF-8"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 93184    0 93184    0     0   383k      0 --:--:-- --:--:-- --:--:--  383k
benutzte HTTP Version: 1.1
 Request : 0 bytes
 Response : 93184 bytes
 Geschwindigkeit : 392755 bytes/s
 Dauer DNS Anfrage : 0.002767 s
 Dauer TCP Aufbau : 0.020922 s
 Dauer TLS Handshake : 0.047238 s
 Gesamtzeit : 0.237257 s
```

HTTP1.1

•



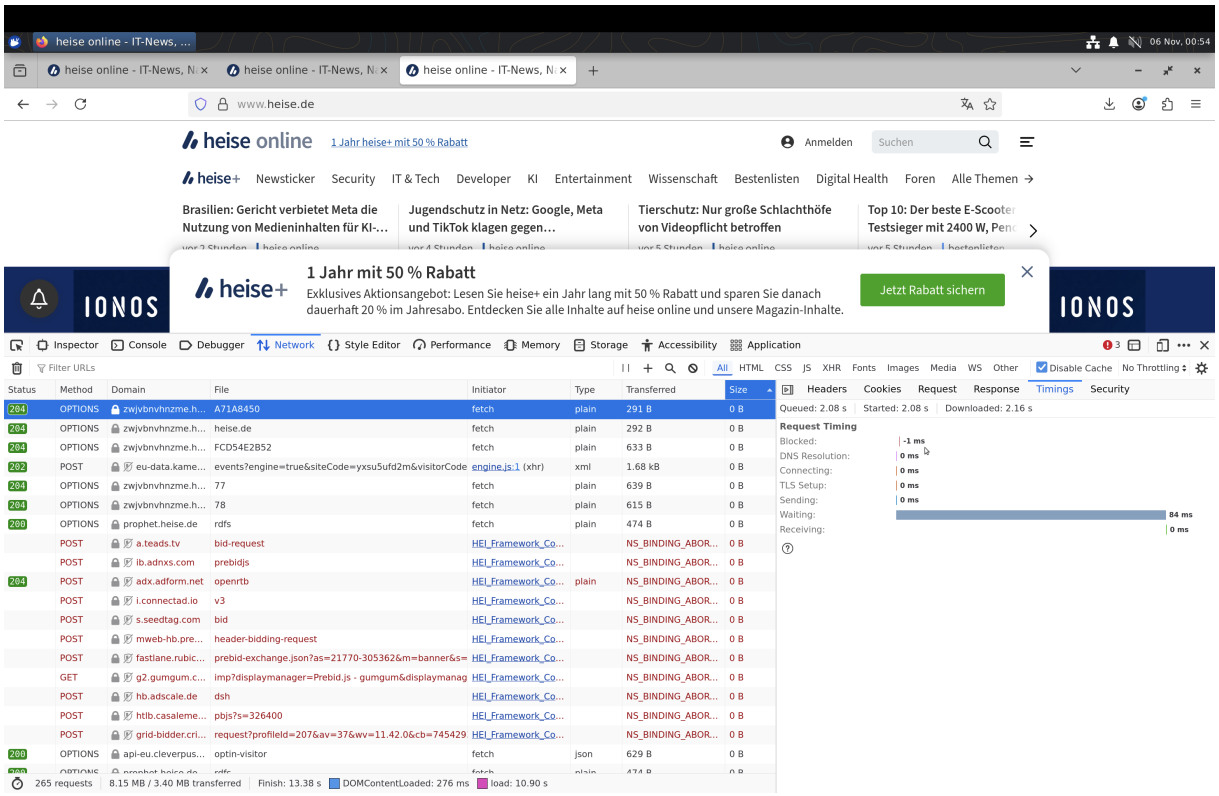
HTTP2.0

Die Anfragen werden mit jedem Protokoll leicht langsamer, da neue Header introduced und genutzt werden. Dadurch wird die benötigte Datenmenge größer und die Zeit wird entsprechend mehr.

```
[melvinlux@Melvins-Macbook-Air tools % ping google.com
 PING google.com (142.251.110.102): 56 data bytes
 64 bytes from 142.251.110.102: icmp_seq=0 ttl=109 time=13.695 ms
 64 bytes from 142.251.110.102: icmp_seq=1 ttl=109 time=41.998 ms
 64 bytes from 142.251.110.102: icmp_seq=2 ttl=109 time=88.894 ms
 64 bytes from 142.251.110.102: icmp_seq=3 ttl=109 time=17.626 ms
 64 bytes from 142.251.110.102: icmp_seq=4 ttl=109 time=14.838 ms
 64 bytes from 142.251.110.102: icmp_seq=5 ttl=109 time=14.212 ms
 64 bytes from 142.251.110.102: icmp_seq=6 ttl=109 time=17.041 ms
 64 bytes from 142.251.110.102: icmp_seq=7 ttl=109 time=17.278 ms
 64 bytes from 142.251.110.102: icmp_seq=8 ttl=109 time=16.486 ms
 64 bytes from 142.251.110.102: icmp_seq=9 ttl=109 time=15.550 ms
 64 bytes from 142.251.110.102: icmp_seq=10 ttl=109 time=17.483 ms
 64 bytes from 142.251.110.102: icmp_seq=11 ttl=109 time=18.431 ms
 64 bytes from 142.251.110.102: icmp_seq=12 ttl=109 time=21.684 ms
 64 bytes from 142.251.110.102: icmp_seq=13 ttl=109 time=62.370 ms
 64 bytes from 142.251.110.102: icmp_seq=14 ttl=109 time=99.613 ms
 64 bytes from 142.251.110.102: icmp_seq=15 ttl=109 time=14.015 ms
 64 bytes from 142.251.110.102: icmp_seq=16 ttl=109 time=17.038 ms
 64 bytes from 142.251.110.102: icmp_seq=17 ttl=109 time=15.554 ms
 64 bytes from 142.251.110.102: icmp_seq=18 ttl=109 time=20.212 ms
 64 bytes from 142.251.110.102: icmp_seq=19 ttl=109 time=16.125 ms
 64 bytes from 142.251.110.102: icmp_seq=20 ttl=109 time=18.293 ms
 64 bytes from 142.251.110.102: icmp_seq=21 ttl=109 time=15.529 ms
 ^C
 --- google.com ping statistics ---
 22 packets transmitted, 22 packets received, 0.0% packet loss
 round-trip min/avg/max/stddev = 13.695/26.998/99.613/23.888 ms
```

Im Durchschnitt beträgt die RTT 26,998ms, demnach ist der Ping ca 10 mal schneller als eine HTTP anfrage. Dies lässt sich begründen damit das ICMP eine geringere Packagemenge benutzt und somit weniger Aufwand auf bieden Seiten der Verbindung ist.

3.5.3 c



Wie im Bild zu erkennen ist, wurden 265 Objekte geladen, was insgesamt 13.38s gedauert hat. Das kleinste Objekt ist 0 Bytes groß mit einer Zeit von 84 ms. Der TCP Verbindungsaufbau hat 0 ms gedauert.

3.5.4 d

Die Daten werden von Wireshark mit Hilfe von ip.addr == 193.99.144.85 gefiltert. Die IP von der Webseite heise.de wurde davor herausgefunden. Folgende Protokolle werden bei heise.de angezeigt: IPv4 auf der Netzwerkschicht, TCP auf der Transportschicht und TLSv1.3 auf der Anwendungsschicht Für google.com werden hingegen IPv4 auf der Netzwerkschicht, UDP auf der Transportschicht und QUIC auf der Anwendungsschicht angezeigt. Der wesentliche Unterschied besteht darin, dass heise.de TCP nutzt, welches einen zeitaufwendigen Handshake für den Verbindungsaufbau und die Verschlüsselung benötigt. Google.com verwendet das QUIC-Protokoll über das verbindungslose UDP, welches die Verschlüsselung direkt integriert.

4 AI-Chats

- https://claude.ai/share/1978160c-67cf-44c7-aac8-82b429b94b13
- https://claude.ai/share/4f568fbc-d6d6-4cfd-9992-4858c2a67744